

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **PubSub System**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- The system should support multiple topics where messages can be published
- Publisher should be able to publish a message to a particular topic
- Subscribers should be able to subscribe to a topic
- Whenever a message is published to a topic, all subscribers subscribed to that topic should receive the message
- The system should support topic management
- The system should maintain notification channels (build scalable system)

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Edge Cases to Consider

- Subscriber goes offline during message delivery or the user is offline?
 - How many types of notifications?
 - What happens when someone comes online after a long time?
-

Step 2: Identify Core Entities

1. Topic (Core Entity)

- id: String [PK]
- name: String
- isActive: boolean
- createdAt: long

2. Subscriber

- id: String [PK]
- email: String
- realtimeConnectionId: String (nullable)
- isOnline: boolean
- createdAt: long
- lastHeartbeat: long

3. Subscription

- id: String [PK]
- topicId: String [FK]
- subscriberId: String [FK]
- isActive: boolean
- createdAt: long

4. Message

- id: String [PK]
- topicId: String [FK]
- content: String
- timestamp: long

5. MessageDelivery

- id: String [PK]
- messageId: String [FK]
- subscriberId: String [FK]
- channel: DeliveryChannel (Enum: EMAIL, REALTIME)
- status: DeliveryStatus (Enum: PENDING, DELIVERED, ACKNOWLEDGED)
- createdAt: long
- acknowledgedAt: long (nullable)

6. DeliveryChannel (Enum)

- EMAIL
- REALTIME

7. DeliveryStatus (Enum)

- PENDING
- DELIVERED
- ACKNOWLEDGED

Interview Tip: Clearly identifying entities and enums at this stage helps define relationships (like Topic ↔ Subscriber, Topic ↔ Message) and ensures smooth implementation of subscription, delivery, and notification mechanisms in the system.

Step 3: Visualize Interaction Flows

1. Topic Management:

- Create topic
- List topics
- Deactivate topic

2. Message Publishing:

- Publish message
- Acknowledge message

3. Subscriber Management:

- Register subscriber
- Go online
- Go offline

4. Subscription Management:

- Subscribe to topic
 - Unsubscribe from topic
-

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Service Layer → Domain Layer

CONTROLLERS:

1. TopicController (Topic Management)

- - Topic createTopic(String name)
- - List<Topic> getAllTopics()
- - void deactivateTopic(String topicId)

2. PublisherController (Message Publishing)

- - Message publishMessage(String topicId, String content)

3. SubscriberController (Subscriber Management)

- - Subscriber registerSubscriber(String email)
- - void goOnline(String subscriberId, String connectionId)
- - void goOffline(String subscriberId)

4. SubscriptionController (Subscription Management)

- - Subscription subscribeToTopic(String topicId, String subscriberId)
- - void unsubscribeFromTopic(String topicId, String subscriberId)

5. MessageController (Message Consumption)

- - void acknowledgeMessage(String messageId, String subscriberId)

SERVICES:

1. TopicService (Topic Management)

- - Topic createTopic(String name)
- - List<Topic> getAllTopics()
- - void deactivateTopic(String topicId)

2. PublisherService (Message Publishing)

- - Message publishMessage(String topicId, String content)
- - void processMessageDeliveryAsync(Message message, Topic topic)

3. SubscriberService (Subscriber Management)

- - Subscriber registerSubscriber(String email)
- - void goOnline(String subscriberId, String connectionId)
- - void goOffline(String subscriberId)
- - void pushPendingDeliveries(String subscriberId)

4. SubscriptionService (Subscription Management)

- - Subscription subscribeToTopic(String topicId, String subscriberId)
- - void unsubscribeFromTopic(String topicId, String subscriberId)

5. MessageService (Message Consumption)

- - void acknowledgeMessage(String messageId, String subscriberId)

REPOSITORIES:

1. TopicRepository

- - Topic save(Topic topic)
- - List<Topic> findAll()
- - Optional<Topic> findById(String topicId)
- - void deleteById(String topicId)

2. SubscriberRepository

- - Subscriber save(Subscriber subscriber)
- - Optional<Subscriber> findById(String subscriberId)
- - List<Subscriber> findAll()
- - void updateOnlineStatus(String subscriberId, boolean isOnline, String connectionId)
- - void deleteById(String subscriberId)

3. SubscriptionRepository

- - Subscription save(Subscription subscription)

- - List<Subscription> findByTopic(String topicId)
- - List<Subscription> findBySubscriber(String subscriberId)
- - void deactivateSubscription(String topicId, String subscriberId)
- - void deleteById(String subscriptionId)

4. MessageRepository

- - Message save(Message message)
- - Optional<Message> findById(String messageId)
- - void deleteById(String messageId)

5. MessageDeliveryRepository

- - MessageDelivery save(MessageDelivery delivery)
- - List<MessageDelivery> findPendingBySubscriber(String subscriberId)
- - void updateDeliveryStatus(String deliveryId, DeliveryStatus status)
- - void deleteById(String deliveryId)

OBSERVER PATTERN IMPLEMENTATION:

1. SubscriberObserver (Observer Interface)

- + void update(Message message)

2. EmailSubscriber (Concrete Observer)

- - email: String
- + void update(Message message) // Send email notification

3. RealtimeSubscriber (Concrete Observer)

- - connectionId: String
- - subscriberId: String
- + void update(Message message) // Send realtime notification

4. MessageSubject (Subject - Observer Pattern Implementation)

- - emailSubscribers: List<SubscriberObserver>
 - - realtimeSubscribers: List<SubscriberObserver>
 - + void addEmailSubscriber(SubscriberObserver subscriber)
 - + void removeEmailSubscriber(SubscriberObserver subscriber)
 - + void addRealtimeSubscriber(SubscriberObserver subscriber)
 - + void removeRealtimeSubscriber(SubscriberObserver subscriber)
 - + void notify(Message message) // *Calls both email and realtime notifications*
 - + void notifyEmailSubscribers(Message message)
 - + void notifyRealtimeSubscribers(Message message)
 - + List<SubscriberObserver> getEmailSubscribers()
 - + List<SubscriberObserver> getRealtimeSubscribers()
-

Step 5: Core Use Cases & Methods

1. Topic Creation Use Case:

- POST /api/topics →
 - TopicController.createTopic() →
 - TopicService.createTopic() →
 - Create topic with empty subscriber lists →
 - Return topic
-

2. Message Publishing Use Case:

- POST /api/topics/{topicId}/messages →
- PublisherController.publishMessage() →
- PublisherService.publishMessage() →
- Validate topic →
- Create message →

- Return message →
 - Background: processMessageDeliveryAsync() →
topic.getMessageSubject().notify() →
 - Notify email subscribers (always) →
 - Notify realtime subscribers (online only)
-

3. Subscriber Registration Use Case:

- POST /api/subscribers →
 - SubscriberController.registerSubscriber() →
 - SubscriberService.registerSubscriber() →
 - Create subscriber (isOnline=true) →
 - Return subscriber
-

4. Subscription Use Case:

- POST /api/subscriptions →
 - SubscriptionController.subscribeToTopic() →
 - SubscriptionService.subscribeToTopic() →
 - Create subscription →
 - Add EmailSubscriber to topic.getMessageSubject() →
 - If online, add RealtimeSubscriber →
 - Return subscription
-

5. Online Status Change Use Case:

- POST /api/subscribers/{id}/online →

- `SubscriberController.goOnline()` →
 - `SubscriberService.goOnline()` →
 - Update status →
 - Add `RealtimeSubscriber` to all topic `messageSubjects` →
 - Push pending deliveries →
 - Return success
-

6. Offline Status Change Use Case:

- `POST /api/subscribers/{id}/offline` →
 - `SubscriberController.goOffline()` →
 - `SubscriberService.goOffline()` →
 - Update status →
 - Remove from all topic `messageSubject` realtime lists →
 - Return success
-

7. Message Acknowledgment Use Case:

- `POST /api/messages/{messageId}/ack` →
 - `MessageController.acknowledgeMessage()` →
 - `MessageService.acknowledgeMessage()` →
 - Update delivery status →
 - Return success
-

Step 6: Apply OOP Principles & Design Patterns

The system design leverages key design patterns and OOP principles to ensure robust state handling, modular structure, and easy scalability.

Design Patterns Used:

- **Repository Pattern** – for data access abstraction
 - **Service Layer Pattern** – for business logic separation
 - **Observer Pattern** – for message/notification system
 - **RESTful API Design** – for clean HTTP endpoints
 - **Controller Separation Pattern** – for separation of concerns
 - **State Pattern** – for task/message state management
 - **Strategy Pattern** – for flexible sorting algorithms
 - **Composite Pattern** – for task/subtask hierarchy
-

OOP Principles Applied:

- **Single Responsibility** – each controller/service handles one specific concern
 - **Open/Closed** – easy to extend with new notification types, states, or delivery mechanisms
 - **Encapsulation** – domain objects encapsulate their data and behavior
 - **Dependency Inversion** – services depend on abstractions, not concrete implementations
 - **Polymorphism** – different subscriber types, states, or strategies can be swapped at runtime
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Subscriber Offline During Delivery** – Email always sent, realtime only to online subscribers; pending realtime deliveries stored for later
 - **Multiple Notification Channels** – Email (always active) and Realtime (online only)
 - **Subscriber Comes Online After Long Time** – Status updated, added to realtime lists, push pending messages, acknowledgments handled, email continues working
 - **Task Deletion with Subtasks** – Cascade delete all subtasks recursively
 - **Concurrent Updates** – Use optimistic locking with version field
 - **Task Priority Conflicts** – Auto-adjust parent task priority upward if child has higher priority
 - **State Transition Rules** – Strict state machine ensures valid transitions
-

Implementation Strategies:

- **Observer Pattern** – Separate lists for email and realtime subscribers per topic/task
 - **Thread Safety** – Synchronized collections for subscriber lists
 - **Message Persistence** – Store messages and delivery records for audit and retry
 - **Online Status** – Track subscriber connection status and heartbeat
 - **Pending Delivery** – Query pending deliveries when subscriber comes online
 - **Acknowledgment** – Only realtime messages require explicit acknowledgment
 - **Dual Channel** – Email always works; realtime only when online
 - **Recursive Subtasks** – Maintain hierarchy consistency and recursive priority updates
-

Summary

This simplified design focuses on 6 core API flows perfect for a 60-minute interview:

- **Topic Creation:** POST /api/topics (core topic management)
- **Message Publishing:** POST /api/topics/{topicId}/messages (publish & notify subscribers)
- **Subscriber Registration:** POST /api/subscribers (user registration & status tracking)
- **Subscription Management:** POST /api/subscriptions (subscribe/unsubscribe to topics)
- **Online/Offline Status Change:** POST /api/subscribers/{id}/online or /offline (realtime notification handling)
- **Message Acknowledgment:** POST /api/messages/{messageId}/ack (delivery confirmation)

Key Features Implemented:

- Dual-channel notification system: Email (always) & Realtime (online only)
- Observer pattern for subscriber notifications (FULLY IMPLEMENTED)
- Persistent message storage and delivery tracking
- Thread-safe handling of subscriber lists
- Optimistic delivery & acknowledgment handling
- Clean architecture with proper separation of concerns
- RESTful API design with clear endpoints
- Comprehensive edge case handling for offline subscribers, pending deliveries, and reconnections
- Subscriber status tracking for efficient realtime message delivery

Design Pattern Highlights:

- **Observer Pattern** – Notifies email & realtime subscribers of new messages (FULLY IMPLEMENTED)
- **Repository Pattern** – Abstracts data access for messages, subscribers, and subscriptions
- **Service Layer Pattern** – Handles business logic separately from controllers
- **RESTful API Design** – Clean and structured HTTP endpoints
- **Controller Separation** – Different controllers for topics, messages, subscriptions, and subscribers
- **Polymorphism** – Email and Realtime subscribers handled uniformly via Observer interface